LLM-First PaddleOCR Evaluator (Qwen via Hugging Face / Nebius)

This tool processes **order-sheet images** with PaddleOCR, then **always** calls **Qwen/Qwen3-4B** through the Hugging Face `InferenceClient` (provider = `nebius`) to extract a clean JSON of **header fields only**.

❗Intentionally **excluded**: product rows, `total_units`, and `total_layers`.

What it does

- Scans a folder of images (default: `./data/`)
- Runs **PaddleOCR** to get raw text lines + boxes
- ALWAYS sends the OCR text to **Qwen/Qwen3-4B** for schema-constrained extraction
- Normalizes common issues (dates, `printed_date` with time, `dock` formatting like `D05`, address punctuation) and removes "ID/Number" residue from `shipment_id` / `asn_number`
- Writes results to CSV
- Evaluates against `annotations.jsonl` (ground truth)
- Exports **raw OCR** and **raw LLM** dumps to JSON for debugging

Output schema (exact keys)

```
route, pallet_number, delivery_date, load, dock,
shipment_id, destination, asn_number, salesman,
total_cases, printed_date, page_number
```

Installation

```
# CPU builds shown; pick the correct paddlepaddle build if using GPU
pip install paddleocr==2.7.0.3 paddlepaddle==2.6.1
pip install huggingface_hub
```

Set your Hugging Face token

You can pass `--hf_token` at run time, or set an env var:

PowerShell (Windows):

```
$env:HF_TOKEN="hf_XXXXXXXXXXXXXXXXXXXXXXXX"
```

CMD (Windows):

```
set HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXX
```

bash/zsh (Linux/macOS):

```
export HF_TOKEN=hf_XXXXXXXXXXXXXXXXXXXXXXXX
```

Verify it's set:

```
echo $env:HF_TOKEN      # PowerShell
echo %HF_TOKEN%         # CMD
echo $HF_TOKEN          # bash/zsh
```

Data layout

- **Images** go in `./data/` (or any folder you pass via `--data_dir`).
- **Ground truth** file: JSON Lines at `./data/annotations.jsonl` (or your path).
  Each line has at least `image` and a JSON payload in `suffix` containing the header fields.
  Example line:
- `{`
- `  "image": "IMG_3023.jpg",`
- ` "suffix": "{\"route\":\"M906-HQ-980\",\"pallet_number\":\"11\",\"delivery_date\":\"5/10/2024\",\"load\":\"5\",\"dock\":\"D20\",\"shipment_id\":\"Z18655822187\",\"destination\":\"Unit 3960 Box 4834, DPO AE 90347\",\"asn_number\":\"7948020197\",\"salesman\":\"JOSHUA JOHNSON\",\"total_cases\":\"90\",\"printed_date\":\"11/29/2024 16:18\",\"page_number\":\"26\"}"`
- `}`

> The script automatically **reduces GT** to header-only fields and **ignores** product rows and unit/layer totals.

How to run

```
python batch_eval_llm_only.py \
  --data_dir ./data \
  --annotations ./data/annotations.jsonl \
  --out_csv ./output/predictions.csv \
  --metrics_json ./output/metrics.json \
  --raw_json ./output/raw_ocr.json \
  --raw_llm_json ./output/llm_preds.json
  # --hf_token hf_XXXX   (optional; otherwise uses HF_TOKEN env)
  # --llm_model Qwen/Qwen3-4B   (optional override)
  # --lang en                   (OCR language, default en)
```

Typical directory tree

```
project/
├─ batch_eval_llm_only.py
```

```
├─ data/
│  ├─ annotations.jsonl
│  └─ *.jpg|png|webp|tif...
└─ output/
   ├─ predictions.csv
   ├─ metrics.json
   ├─ raw_ocr.json
   └─ llm_preds.json
```

What you get

`output/predictions.csv`

Per-image fields + accuracy:

- Columns: `image` + the 12 header fields + `field_accuracy` + `overall_accuracy`

`output/metrics.json`

Summary numbers:

```
{
  "num_images": 150,
  "num_with_ground_truth": 150,
  "mean_field_accuracy": 0.97,
  "mean_overall_accuracy": 0.97
}
```

`output/raw_ocr.json`

All OCR entries (text + geometry) per image for debugging.

`output/llm_preds.json`

Raw LLM response text (as returned by Nebius, minus any `<think>` sections during parsing) and the parsed/normalized JSON used in the CSV.

How evaluation works

- The model's final **LLM output** (after normalization/sanitization) is compared to ground truth.
- For each field in the schema:
  - It's counted **correct** if the **normalized** prediction equals the **normalized** GT **and** the GT field is **non-empty**.
  - Otherwise it counts 0 for that field.
- `field_accuracy` = mean across all fields.
- `overall_accuracy` = same as `field_accuracy` (header-only evaluation).

Normalization rules (high level):

- `delivery_date` → `MM/DD/YYYY`
- `printed_date` → `MM/DD/YYYY HH:MM` (joins date/time if OCR splits them)
- `dock` → canonicalized like `D05`
- `destination` → punctuation-insensitive comparison (commas/periods/spaces collapsed)
- Numeric-only where appropriate (`total_cases`, `page_number`, `pallet_number`)
- `shipment_id` / `asn_number` → removes stray "ID" / "Number" words

If you prefer to **ignore** empty GT fields (not count them toward the average), you can adjust `compare_header_fields` in the script.

Customization tips

- **Model**: change with `--llm_model` (defaults to `Qwen/Qwen3-4B`).
- **OCR language**: pass `--lang` (default `en`).
- **Provider**: the code is wired to `provider="nebius"`. If you switch providers, ensure your token and endpoint support `chat.completions`.

Troubleshooting

- `PaddleOCR import failed`:
  Ensure both `paddleocr` and a compatible `paddlepaddle` build are installed for your platform (CPU/GPU).
- `FATAL: No HF token`:
  Pass `--hf_token` or set the `HF_TOKEN` environment variable.
- Empty outputs / low accuracy:
  - Verify images exist and are readable.
  - Check OCR text in `raw_ocr.json` to see if the page text is sensible.
  - Inspect `llm_preds.json` to see the raw LLM content and parsed JSON.

Privacy note

Only the **OCR text** is sent to Hugging Face Inference (Nebius). Images are **not** uploaded by this script.